

Mes notes de départ

TechDay | Construire une base de connaissances avec un chatbot

« Donc j'avais écrit avec des dessins le but de ce travail. »

Le but reconstitué

Créer une application locale simple à ouvrir dans un navigateur. Elle sert à consulter une base de connaissances personnelle et à poser des questions à un chatbot qui répond à partir de cette base.

- Interface locale : une page HTML avec un onglet fiches et un onglet chatbot.
- Données organisées : un CSV de départ importé dans une table Supabase.
- IA utile : Gemini répond après avoir reçu les fiches pertinentes.
- Mémoire du projet : AGENTS.md garde les règles et GitHub garde les versions.

Phrase de départ à donner à ChatGPT

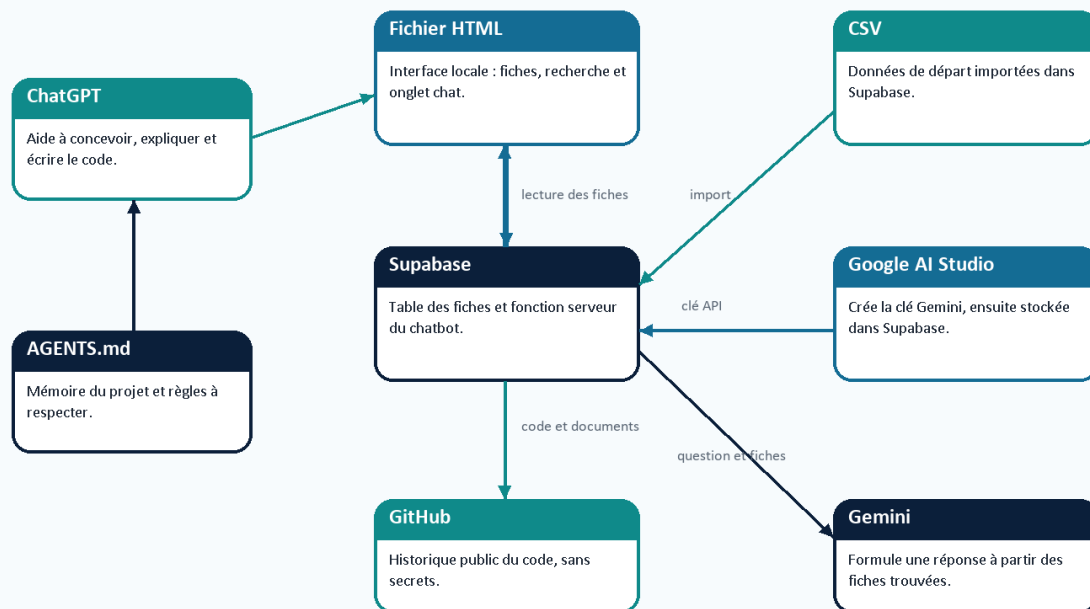
« Je veux une application HTML locale pour consulter ma base de connaissances et poser des questions à un chatbot. Les données sont dans Supabase, la clé Gemini reste dans Supabase, et GitHub sert à conserver l'historique du code. Explique-moi les étapes simplement. »

Vue d'ensemble du système

Les outils et leurs liaisons

Architecture de la base de connaissances TechDay

Les flèches montrent les échanges utiles au fonctionnement.



Architecture actuelle de l'application TechDay

Le navigateur lit les fiches depuis Supabase. Quand une question est posée, il appelle une fonction Supabase. Cette fonction sélectionne des fiches pertinentes puis demande à Gemini de rédiger une réponse. La clé Gemini ne passe jamais dans le navigateur.

Rôle de chaque élément

Comprendre ce qui sert à quoi

Élément	Rôle	Lien principal
HTML	La page locale visible dans le navigateur.	Lit les fiches et envoie les questions.
CSV	La source de départ des fiches.	Est importé dans Supabase sans modifier l'original.
Supabase	La base de données et le serveur du chatbot.	Conserve fiches, secrets et fonction Edge.
Google AI Studio	Crée la clé Gemini.	La clé est stockée dans les secrets Supabase.
Gemini	Formule une réponse à partir du contexte.	Reçoit les fiches sélectionnées, sans URL.
AGENTS.md	Mémoire et règles du projet.	Cadre les demandes et les décisions.
GitHub	Historique des versions du code.	Reçoit le code et les documents sans secrets.
ChatGPT	Aide à concevoir, coder et expliquer.	Lit AGENTS.md et suit les demandes.

Règle essentielle

La clé Gemini est secrète. Elle ne doit jamais être collée dans le fichier HTML, dans le CSV ou dans GitHub. Le navigateur ne contient qu'une clé Supabase publishable, conçue pour être visible côté application.

Étape 1 Clarifier le projet

Partir d'une demande claire et d'une mémoire durable

Avant de créer du code, il faut définir ce que l'outil doit faire, où les données vivent et où les secrets restent cachés. AGENTS.md sert de mémoire : il évite d'oublier les règles importantes à la prochaine séance.

1. Créer la règle de travail

Demande à formuler

« Crée un AGENTS.md en français. Il doit rappeler que le CSV original ne doit pas être modifié, que les clés secrètes ne vont jamais dans GitHub, et que chaque étape vérifiée doit être notée dans JOURNAL.md et ROADMAP.md. »

Résultat attendu

Un fichier de règles court, lisible et réutilisable à chaque séance.

2. Décrire le résultat attendu

Demande à formuler

« Je veux une page HTML locale avec deux onglets : Base de connaissances et Chatbot. Supabase contient les fiches. Gemini répond à partir des fiches. GitHub conserve le code. »

Résultat attendu

Une architecture simple : interface, données, IA et historique sont séparés.

Pourquoi cette étape est utile

Une demande précise évite que l'application mélange les clés, les données et le code. Elle indique aussi ce qui doit rester local, public ou secret.

Étape 2 Préparer les données

Respecter le CSV et structurer Supabase

3. Analyser le fichier de départ

Demande à formuler

« Analyse le CSV Base de connaissance sans le modifier. Indique le nombre de lignes, les colonnes, les lignes vides et les champs manquants. Prépare un plan d'import dans Supabase. »

Résultat attendu

Un inventaire clair. Ici, la table contient 147 fiches, dont certaines sont incomplètes ou sans titre.

4. Créer la table

Demande à formuler

« Crée dans Supabase une table base_connaissances avec les colonnes du CSV. Garde l'identifiant de chaque fiche et active RLS. N'efface aucune donnée existante. »

Résultat attendu

Une table qui reçoit les fiches et peut évoluer plus tard.

Résultat actuel

- Les 147 fiches du CSV sont dans Supabase.
- L'application les affiche en lecture seule.
- Les fiches sans titre restent visibles comme « Fiche sans titre » : elles ne sont pas supprimées.
- La recherche de la page HTML cherche dans les titres, notes, textes et étiquettes.

Étape 3 Construire l'interface HTML

Créer une application locale simple

5. Créer la page

Demande à formuler

« Crée un index.html moderne et lisible. Il doit se connecter à Supabase avec une clé publishable, afficher les fiches, proposer une recherche exacte et avoir un deuxième onglet Chatbot. Il ne doit contenir aucune clé Gemini. »

Résultat attendu

Une page autonome ouverte localement dans un navigateur.

La page HTML est le comptoir de l'application : elle affiche les données et transmet la question au serveur Supabase. Elle ne répond pas elle-même avec de l'IA.

Connexion ou pas

Le choix actuel est une consultation sans connexion. Cela rend les fiches accessibles en lecture à toute personne qui peut atteindre le projet Supabase. Si les fiches deviennent sensibles, la bonne demande à formuler est : « Remets une connexion Supabase et autorise seulement mon compte à lire les fiches. »

Le chatbot RAG

Étape 4 Faire répondre l'IA à partir des fiches

RAG signifie Retrieval Augmented Generation. En français : le chatbot cherche d'abord des documents, puis il donne ces documents à l'IA pour rédiger une réponse. Cela limite les réponses inventées et permet d'afficher les sources.

6. Créer la fonction chatbot

Demande à formuler

« Crée une Edge Function Supabase chat-with-knowledge. Elle reçoit une question, cherche des fiches pertinentes dans base_connaissances, en transmet au maximum 12 à Gemini et retourne une réponse en français avec la liste des fiches utilisées. N'envoie pas les URL à Gemini. Utilise le secret API_1. »

Résultat attendu

Un chatbot côté serveur : la clé Gemini ne quitte pas Supabase et la réponse garde une trace des fiches utilisées.

Le RAG actuel

- Recherche : les mots de la question sont comparés aux textes des fiches.
- Contexte : au plus 12 fiches correspondantes sont données à Gemini.
- Réponse : Gemini formule une réponse courte et indique les fiches utilisées.
- Limite : une question formulée avec des mots très différents peut ne pas trouver la bonne fiche.

Étape 5 Créer et protéger la clé Gemini

Google AI Studio puis secrets Supabase

7. Préparer la clé

Demande à formuler

« Explique-moi comment créer une clé Gemini dans Google AI Studio, puis comment la stocker dans les secrets Supabase avec le nom API_1. Ne me demande jamais de la coller dans le HTML ou dans GitHub. »

Résultat attendu

Une clé créée dans Google AI Studio et conservée uniquement par Supabase.

Pourquoi passer par Supabase

Si la clé est placée dans le HTML, toute personne qui ouvre la page peut la récupérer et utiliser ton quota. Dans une Edge Function, la clé reste côté serveur. Le navigateur demande seulement une réponse au chatbot.

Outils IA à connaître

Outil	À quoi il sert	Quand le demander
Gemini API	Rédiger une réponse ou produire des résumés.	Quand les fiches utiles ont déjà été trouvées.
Edge Function Supabase	Exécuter du code côté serveur et accéder au secret API_1.	Quand une action nécessite une clé secrète.
pgvector	Comparer le sens des textes avec des vecteurs.	Quand la recherche par mots devient insuffisante.
Plugin ou connecteur	Relier un outil externe à l'assistant.	Seulement lorsqu'un service extérieur est nécessaire.

Rendre l'IA plus performante

Passer du RAG simple au RAG par le sens

1. Améliorer les fiches avant tout

- Donner un titre clair à chaque fiche, même lorsqu'elle contient seulement une URL.
- Ajouter une phrase qui explique la valeur de la ressource.
- Utiliser des étiquettes cohérentes : IA, logiciel, matériel, artiste, tutoriel.
- Repérer les doublons et les fiches vides, mais conserver le CSV original intact.

2. Ajouter la recherche par le sens

Avec pgvector, chaque fiche reçoit une représentation numérique de son sens, appelée embedding. Une question comme « animation 3D » peut alors trouver une fiche sur Blender même si elle ne contient pas exactement les mêmes mots.

8. Demande pour la version suivante

Demande à formuler

« Ajoute pgvector dans Supabase. Crée des embeddings pour le titre, le texte et les étiquettes de chaque fiche. Le chatbot doit combiner recherche par mots et recherche par le sens, puis citer les fiches utilisées. »

Résultat attendu

Un RAG plus tolérant aux synonymes et aux formulations naturelles.

3. Ajouter un agent d'enrichissement avec prudence

Un agent peut compléter une fiche en lisant une source fiable. Il doit enregistrer l'URL de la source, proposer les informations manquantes et signaler les cas où il ne peut pas vérifier. Il ne doit pas inventer de contenu.

Étape 6 Tester et garder l'historique

Vérifier avant de publier une version

9. Tester le système

Demande à formuler

« Vérifie que les 147 fiches se chargent, que la recherche exacte fonctionne, que le chatbot répond à une question sur Blender et qu'il affiche la source utilisée. Note le résultat dans JOURNAL.md. »

Résultat attendu

Une preuve que la base, le HTML et le chatbot travaillent ensemble.

10. Envoyer le code dans GitHub

Demande à formuler

« Utilise GitHub CLI pour me donner un lien et un code de connexion. Après mon autorisation, crée un commit clair, récupère l'historique existant sans rien écraser, puis envoie le code sans force push. »

Résultat attendu

Un dépôt GitHub avec les versions du code et des documents de suivi.

Ce qui doit aller dans GitHub

- index.html, les fonctions Supabase, les scripts et les documents de suivi.
- AGENTS.md, JOURNAL.md, ROADMAP.md et DECISIONS.md.
- Jamais : API_1, mots de passe, clé service_role, export de secrets ou CSV confidentiel.

Checklist avant la prochaine évolution

- Formuler une demande claire et la faire relire par rapport à AGENTS.md.
- Tester sur une copie ou vérifier les données existantes avant une modification Supabase.
- Vérifier que la clé API reste dans Supabase.
- Mettre à jour le journal et envoyer un commit avec un message qui décrit le changement.